



SCHOOL OF COMPUTING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

UNIT-I S11BLH21 - Programming in python

INTRODUCTION

History of Python- Introduction to the IDLE interpreter (shell) – Data Types - Built-in function
- Conditional statements - Iterative statements- Input/output Functions- Python database
communication – Data analysis visualization using python

1. Program

A program performs a task in the computer. But, in order to be executed, a program must be written in the machine language of the processor of a computer. Unfortunately, it is extremely difficult for humans to read or write a machine language program. This is because a machine language is entirely made up of sequences of bits. However, high level languages are close to natural languages like English and only use familiar mathematical characters, operators and expressions. Hence, people prefer to write programs in high level languages like C, C++, Java, or Python. A high level program is translated into machine language by translators like compiler or interpreter.

a. About Python

Python is a high level programming language that is translated by the python **interpreter**. As is known, an interpreter works by translating line-by-line and executing. It was developed by Guido-van-rossum in 1990, at the National Research Institute for Mathematics and Computer Science in Netherlands. Python doesn't refer to the snake but was named after the famous British comedy troupe, Monty Python's Flying Circus.

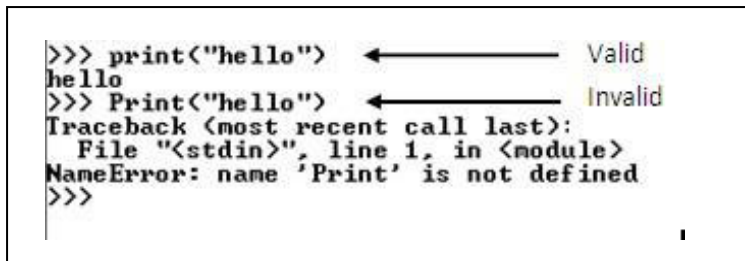
The following are some of the features of Python:

- Python is an Open Source: It is freely downloadable, from the link <http://python.org/>
- Python is portable: It runs on different operating systems / platforms
- Python has automatic memory management
- Python is flexible with both procedural oriented and object oriented programming
- Python is easy to learn, read and maintain

It is very flexible with the console program, Graphical User Interface (GUI) applications, Web related programs etc.

POINTS TO REMEMBER WHILE WRITING A PYTHON PROGRAM

- Case sensitive : Example - In case of print statement use only lower case and not upper case, (See the snippet below)



```
>>> print("hello") ← Valid
hello
>>> Print("hello") ← Invalid
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'Print' is not defined
>>>
```

Fig1. Print Statement

- Punctuation is not required at end of the statement
- In case of string use single or double quotes i.e. ' ' or " "
- Must use proper indentation: The screen shots given below show, how the value of "i" behaves with indentation and without indentation.

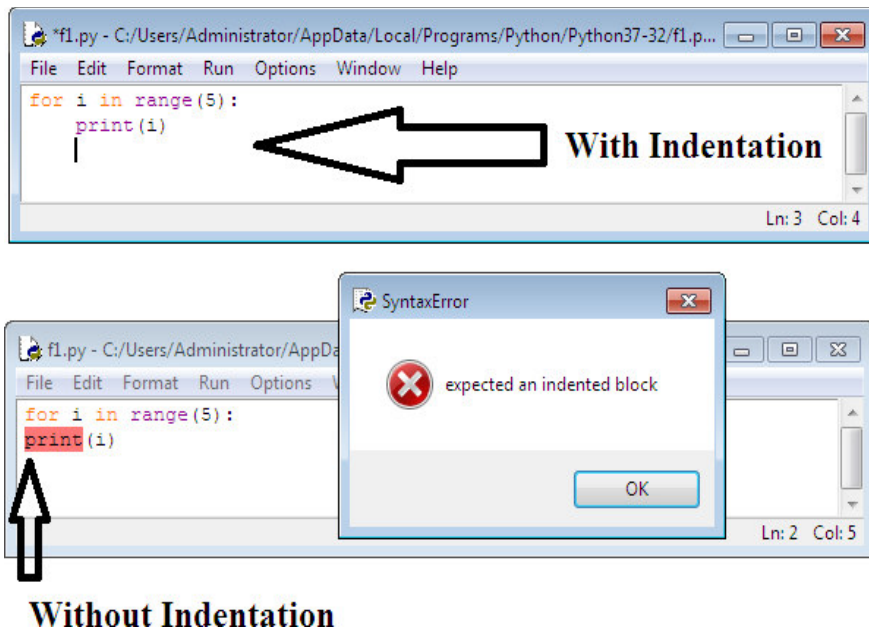


Fig 2 indentation

- Special characters like (,),# etc.are used
- () ->Used in opening and closing parameters of functions
- #-> The Pound sign is used to comment a line

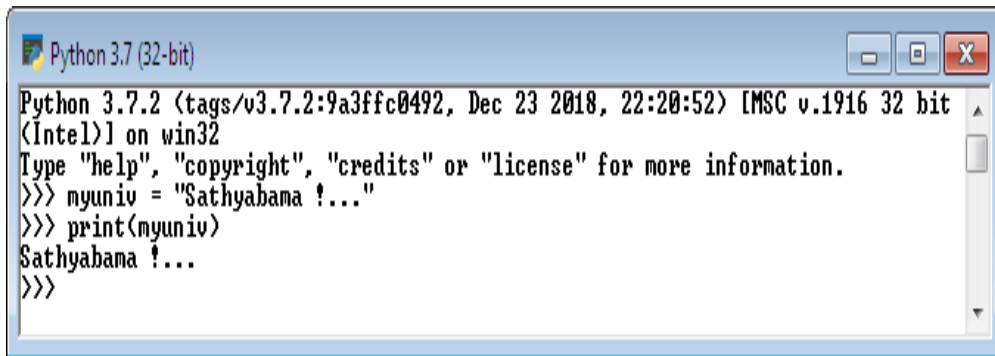
1.2 TWO MODES OF PYTHON PROGRAM

Python Program can be executed in two different modes:

- Interactive mode programming
- Script mode programming

Interactive Mode Programming

It is a command line shell which gives immediate output for each statement, while keeping previously fed statements in active memory. This mode is used when a user wishes to run one single line or small block of code. It runs very quickly and gives instant output. A sample code is executed using interactive mode as below.

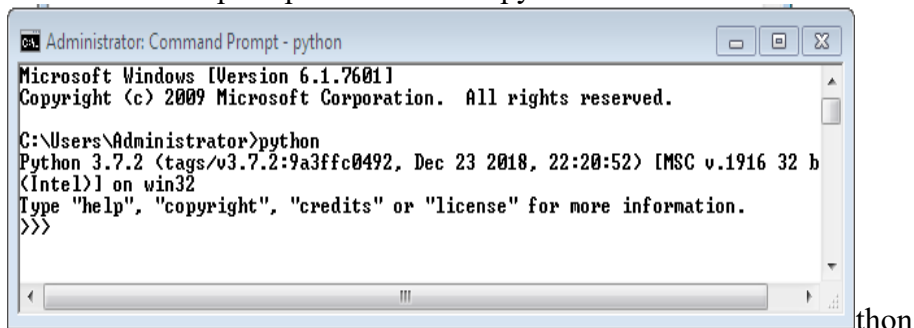


```
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22:20:52) [MSC v.1916 32 bit  
<Intel>] on win32  
Type "help", "copyright", "credits" or "license" for more information.  
>>> myuniv = "Sathyabama !..."  
>>> print(myuniv)  
Sathyabama !...  
>>>
```

Fig 4 sample code is executed using interactive mode

Interactive mode can also be opened using the following ways:

- i) From command prompt c :> users\\...>py



```
Administrator: Command Prompt - python  
Microsoft Windows [Version 6.1.7601]  
Copyright (c) 2009 Microsoft Corporation. All rights reserved.  
  
C:\Users\Administrator>python  
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22:20:52) [MSC v.1916 32 b  
<Intel>] on win32  
Type "help", "copyright", "credits" or "license" for more information.  
>>>  
  
thon
```

Fig 5 interactive mode

The symbol “>>>” in the above screen indicates that the Python environment is in interactive mode.

ii) From the start menu select Python (As shown below)



Fig 6 steps in opening

Script Mode Programming

When the programmer wishes to use more than one line of code or a block of code, script mode is preferred. The Script mode works the following way:

- i) Open the Script mode
- ii) Type the complete program. Comment, edit if required.
- iii) Save the program with a valid name.
- iv) Run
- v) Correct errors, if any, Save and Run until proper output

The above steps are described in detail below:

- i)* To open script mode, select the menu “**IDLE (Python 3.7 32-bit)**” from start menu

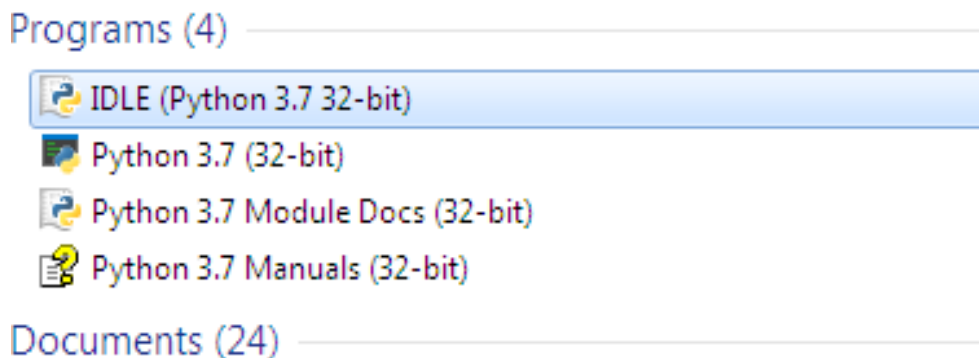


Fig 7 IDLE (Python 3.7 32-bit)

- ii)* After clicking on the menu “**IDLE (Python 3.7 32-bit)**”, a new window with the text Python 3.7.2 Shell will be opened as shown below:

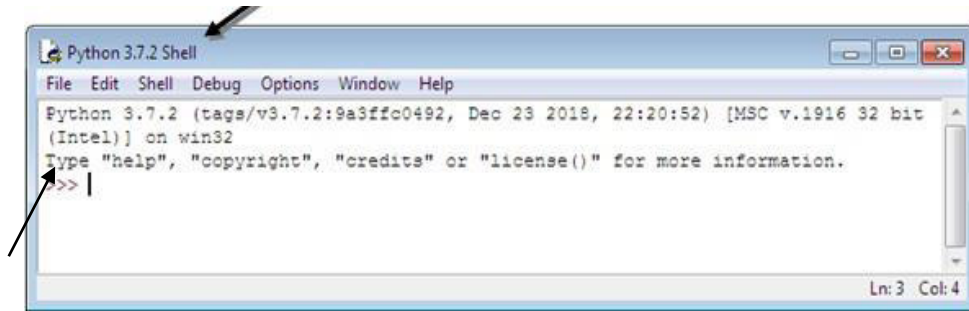


Fig 8 Python 3.7.2 Shell

- iii) Select File → New, to open editor. Type the complete program.
- iv) Select File again; Choose Save.

This will automatically save the file with an extension “.py”.

- v) Select Run → Run Module or Short Cut Key **F5** (*As shown in the screen below*)

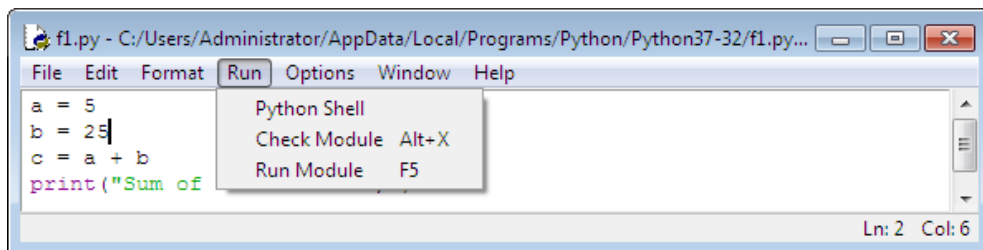


Fig 9. Run Module or Short Cut Key F5

The output of the program will be displayed as below:

```
>>> Sum of a and b is: 30
```

VARIABLES:

Variable is the name given to a reserved memory locations to store values. It is also known as Identifier in python.

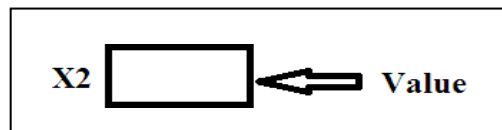
Need for variable:

Sometimes certain parameters will take different values at different time. Hence, in order to know the current value of such parameter we need to have a temporary memory which is identified by a name that name is called as variable. For example, our surrounding temperature changes frequently. In order to know the temperature at a particular time, we need to have a variable.

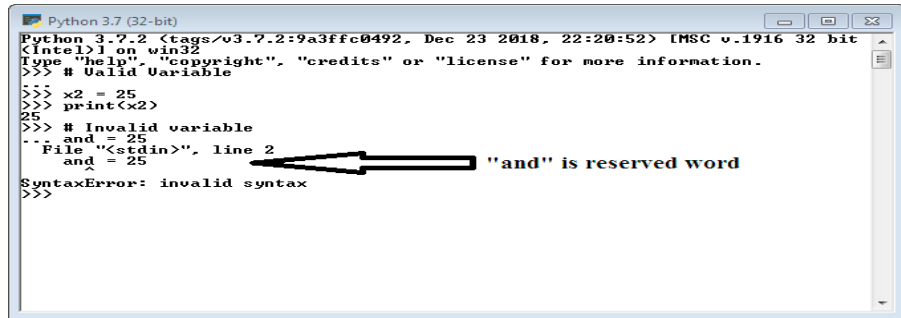
Naming and Initialization of a variable

1. A variable name is made up of alphabets (Both upper and lower cases) and digits
 2. No reserved words
 3. Initialize before calling
 4. Multiple variables initialized
 5. Dynamic variable initialization
- i. Consist of upper and lower case alphabets, Numbers (0-9). E.g. X2

In the above example, a memory space is assigned to variable X2. The value of X2 is stored in this space.



- ii. Reserved words should not be used as variables names.



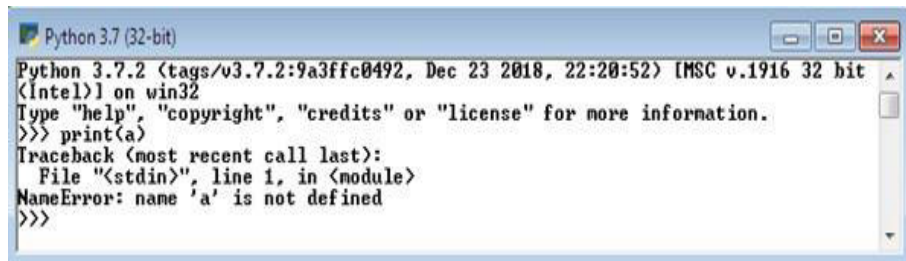
```
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22:20:52) [MSC v.1916 32 bit  
<Intel>] on win32  
Type "help", "copyright", "credits" or "license" for more information.  
>>> # Valid Variable  
>>> x2 = 25  
>>> print(x2)  
25  
>>> # Invalid variable  
... and = 25  
File "<stdin>", line 2  
    and = 25  
SyntaxError: invalid syntax
```

A black arrow points from the text **"and" is reserved word** to the `and = 25` line in the code.

Fig 10. “and” is a reserved word

In the above example “and” is a reserved word, which leads to Syntax error

- iii. Variables must be initialized before it called , else it reports “is not defined ” error message as below E.g.: a=5 print(a)

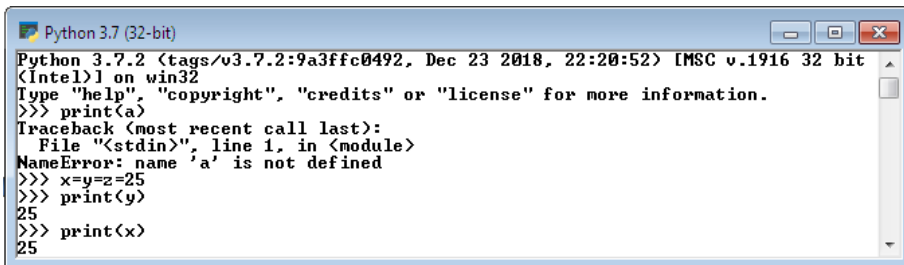


```
Python 3.7 (32-bit)  
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22:20:52) [MSC v.1916 32 bit  
<Intel>] on win32  
Type "help", "copyright", "credits" or "license" for more information.  
>>> print(a)  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
NameError: name 'a' is not defined  
>>>
```

Fig 11 “a” is called before it initialized

In the above example “a” is called before it is initialized. Hence, the python interpreter generates the error message: `NameError: 'a' is not defined`.

- iv. Multiple variables can be initialized with a common value.
E.g.: `x=y=z=25`

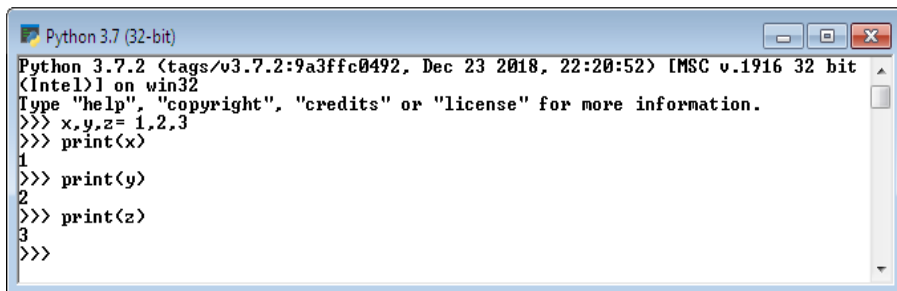


```
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22:20:52) [MSC v.1916 32 bit
<Intel>] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print(a)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'a' is not defined
>>> x=y=z=25
>>> print(y)
25
>>> print(x)
25
```

Fig 12 Multiple variables

In the above three variables x,y,z is assigned with same value 25.

- v. Python also supports dynamic variable initialization. E.g.: `x,y,z=1,2,3`



```
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22:20:52) [MSC v.1916 32 bit
<Intel>] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> x,y,z=1,2,3
>>> print(x)
1
>>> print(y)
2
>>> print(z)
3
>>>
```

Fig 13 dynamic variable initialization

Proper spacing should be given

- `print (10+20+30)` → bad style
- `print (20 + 30 + 10)` → good style

Expression:

An expression is a combination of variables, operators, values and calls to functions. Expressions need to be evaluated.

Need for Expression:

Suppose if you wish to calculate area. Area depends on various parameters in different situations. E.g. Circle, Rectangle and so on...

In order to find area of circle, the expression $\pi * r * r$ must be evaluated and for the rectangle the expression is $w * l$ in case of rectangle. Hence, in this case a variable / value / operator are not enough to handle such situation. So expressions are used. Expression is the combination of variables, values and operations.

A simple example of an expression is $10 + 15$. An expression can be broken down into operators and operands. Few valid examples are given below.

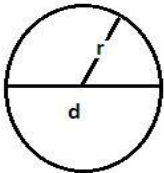
Circle

Solution For Area ▾

$A = \pi r^2$

r Radius

Enter value



Rectangle

Solve for area ▾

$A = w l$

l Length

Enter value

w Width

Enter value

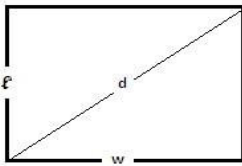
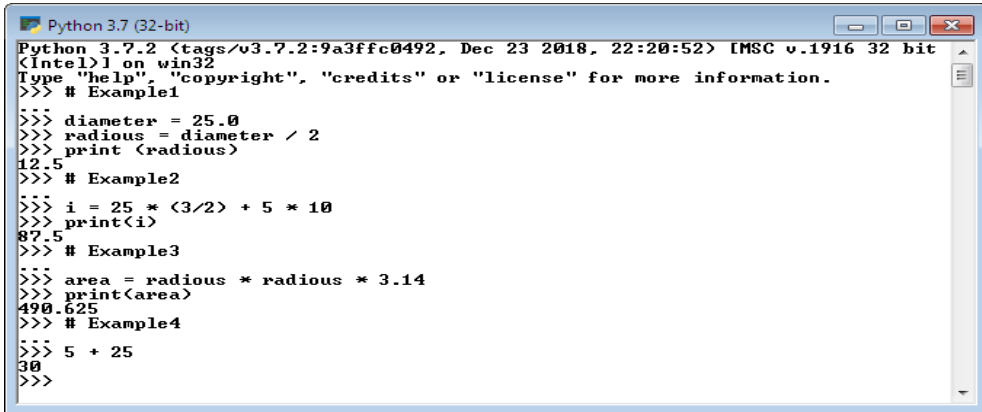


Fig 14 expression

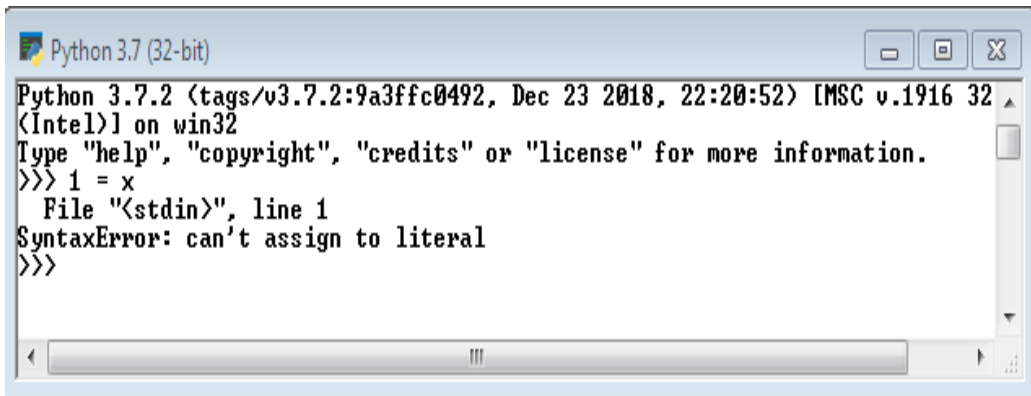


```
Python 3.7 (32-bit)
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22:20:52) [MSC v.1916 32 bit
<Intel>] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> # Example1
>>> diameter = 25.0
>>> radius = diameter / 2
>>> print(radius)
12.5
>>> # Example2
>>> i = 25 * (3/2) + 5 * 10
>>> print(i)
87.5
>>> # Example3
>>> area = radius * radius * 3.14
>>> print(area)
490.625
>>> # Example4
>>> 5 + 25
30
>>>
```

Fig 15 expression

Invalid Expression:

Always values should be assigned in the right hand side of the variable, but in the below example ,the value is given in the left hand side of the variable, which is an invalid syntax for expression.



```
Python 3.7 (32-bit)
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22:20:52) [MSC v.1916 32
<Intel>1 on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 1 = x
      File "<stdin>", line 1
SyntaxError: can't assign to literal
>>>
```

Fig 16 invalid Expression

Data Types:

A Data type indicates which type of value a variable has in a program. However a python variables can store data of any data type but it is necessary to identify the different types of data they contain to avoid errors during execution of program. The most common data types used in python are str(string), int(integer) and float (floating-point).

Strings: Sequence of characters inside single quotes or double quotes.

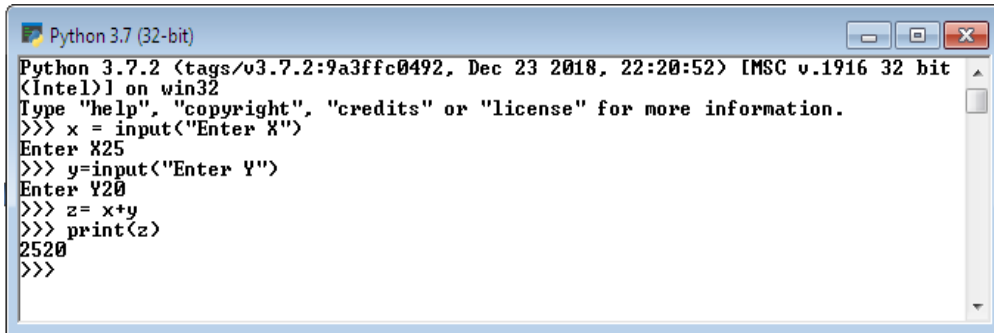
E.g. myuniv = "Sathyabama !.."

Integers: Whole number values such as 50, 100,-3

Float: Values that use decimal point and therefore may have fractional point
E.g.: 3.415, -5.15

By default when a user gives input it will be stored as string. But strings cannot be used for performing arithmetic operations. For example while attempting to perform arithmetic operation add on string values it just

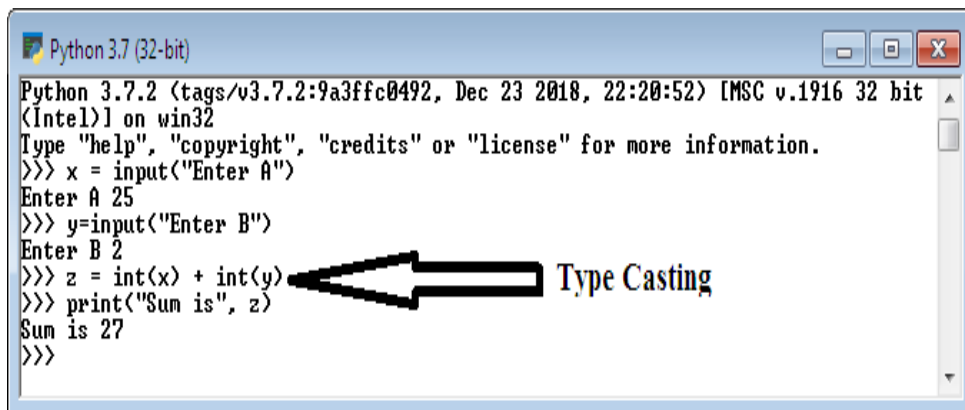
concatenates (joins together) the values together rather performing addition. For example : '25' + '20' = '45' (As in the below Example)



```
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22:20:52) [MSC v.1916 32 bit
<Intel>] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> x = input("Enter X")
Enter X25
>>> y=input("Enter Y")
Enter Y20
>>> z = x+y
>>> print(z)
2520
>>>
```

Fig 17 arithmetic operation add on string values

Fortunately python have an option of converting one data type into another data type (Called as “Casting”) using build in functions in python. The build function `int()` converts the string into integer before performing operation to give the right answer. (As in the below Program)



```
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22:20:52) [MSC v.1916 32 bit
<Intel>] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> x = input("Enter A")
Enter A 25
>>> y=input("Enter B")
Enter B 2
>>> z = int(x) + int(y)
>>> print("Sum is", z)
Sum is 27
>>>
```

Type Casting

Fig 18 type casting

Compound Data Types in Python:

i) List

The List is an ordered sequence of data items. It is one of the flexible and very frequently used data type in Python. All the items in a list are not necessary to be of the same data type.

Declaring a list is straight forward methods. Items in the list are just separated by commas and enclosed within brackets [].

```
>>> list1 =[3.141, 100, 'CSE', 'ECE', 'IT', 'EEE']
```

Table 1 Methods used in list

list1.append(x)	To add item x to the end of the list “list1”
list1.reverse()	Reverse the order of the element in the list “list1”
list1.sort()	To sort elements in the list
list1.reverse()	To reverse the order of the elements in list1.

ii) Tuple

Tuple is also an ordered sequence of items of different data types like list. But, in a list data can be modified even after creation of the list whereas Tuples are immutable and cannot be modified after creation.

The advantages of tuples is to write-protect data and are usually very fast when compared to lists as a tuple cannot be changed dynamically.

The elements of the tuples are separated by commas and are enclosed inside open and closed brackets.

```
>>> t = (50,'python', 2+3j)
```

Table 2 Tuple

List	Tuple
<pre>>>> list1[12,45,27] >>> list1[1] = 55 >>> print(list1) >>> [12,55,27]</pre>	<pre>>>> t1 = (12,45,27) >>> t1[1] = 55 >>> Generates Error Message# Because Tuples are immutable</pre>

i) Set

The Set is an unordered collection of unique data items. Items in a set are not ordered, separated by comma and enclosed inside { } braces. Sets are helpful in performing operations like union and intersection. However, indexing is not done because sets are unordered.

Table 3 Set

List	Set
<pre>>>> L1 = [1,20,25] >>> print(L1[1]) >>> 20</pre>	<pre>>>> S1= {1,20,25,25} >>> print(S1) >>> {1,20,25} >>> print(S1[1]) >>>Error , Set object does not support indexing.</pre>

List Vs Set

ii) Dictionary

The Python Dictionary is an unordered collection of key-value pairs. Dictionaries are optimized for retrieving data when there is huge volume of data. They provide the key to retrieve the value.

In Python, dictionaries are defined within braces {} with each item being a pair in the form key:value. Key and value can be of any type.

```
>>> d1={1:'value','key':2}

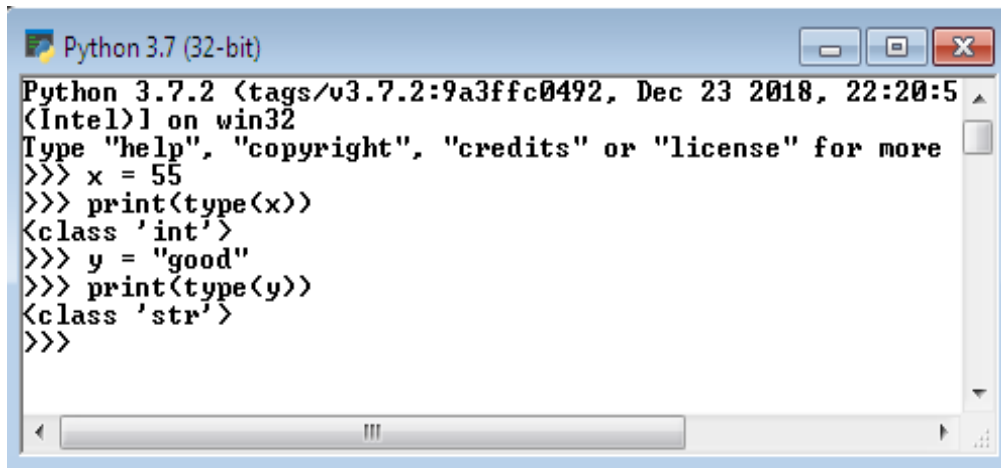
>>> type(d)
```

Table 4 Python’s built-in data type conversion functions

Function	Description	Out Put
int(x)	Converts x into integer whole number	>>>a = int(input("Enter a")) >>>b = int(input("Enter b")) >>>c = a + b >>>print("Sum is ",c)

Function	Description	Out Put
float(x)	Converts x into floating-point number	>>> x = 5 >>> print(float(5)) >>> 5.0
str(x)	Converts x into a string representation	>>> x = 30 >>> y = 70 >>> z = str(x) + str(y) >>> print(z) >>> 3070
chr(x)	Converts integer x into a character	>>> x = 65 >>> print(chr(x)) >>> A >>>
hex(x)	Converts integer x into a hexadecimal string	>>> x = 14 >>> print(hex(x)) >>> 0xe
oct(x)	Converts integer x into an octal string	>>> x = 9 >>> print(oct(x)) >>> 0o11

However to identify the data type of a variable, an in-built python function “type ()” is used. (Example Below)



```

Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22:20:5
<Intel>] on win32
Type "help", "copyright", "credits" or "license" for more
>>> x = 55
>>> print(type(x))
<class 'int'>
>>> y = "good"
>>> print(type(y))
<class 'str'>
>>>
  
```

Fig 19 in-built python function “type ()”

Table5 Python Built-in Functions

Simple Functions		
Function	Description	Output
abs()	Return the absolute value of a number. The argument may be an floating point number or a integer.	<pre>>>> a = -10 >>> print(abs(a)) >>> 10</pre>
max()	Returns the largest number from the list of numbers	<pre>>>> max(12,20,30) >>> 30</pre>
min()	Returns the smallest number from the list of numbers	<pre>>>> min(12,20,30) >>> 12</pre>
pow()	Returns the power of the given number	<pre>>>> pow(5,2) >>>25</pre>
round()	It rounds off the number to the nearest integer.	<pre># E.g. 1: >> round(4.5)</pre>

		>> 5 # Eg 2 >> round(4.567,2) >> 4.57
Mathematical Functions (Using math module)		
ceil(x)	It rounds x up to its nearest integer and returns that integer	>>math.ceil(2.3) >> 3 >>math.ceil(-3.3) >> -3
floor(x)	It rounds x down to its nearest integer and returns that integer	>>math.floor(3.2) >> 3 >>math.floor(-3.4) >> -4
Function	Description	Example
cos(x)	Returns the cosine of x , where x represents angle in radians	>> math.cos(3.14159/2) >> 0 >> math.cos(3.14159) >> -1
sin(x)	Returns the sine of x, where x represents angle in radians	>> math.sin(3.14159/2) >> 1 >> math.sin(3.14159) >> 0
exp(x)	Returns the exponential of x to the base 'e'. i.e. e^x	>> math.exp(1) >> 2.71828
log(x)	Returns the logarithm of x for the base e (2.71828)	>>> math.log(2.71828) >>> 1
log(x,b)	Returns the logarithm of x for the specified base b.	>>> math.log(100,10) >>> 2
sqrt(x)	Returns the square root of x	>>> math.sqrt(16) >>> 4

Note: To include the math module, use the following command:

import math

Conditional Statements

When there is no condition placed before any set of statements, the program will be executed in sequential manure. But when some condition is placed before a block of statements the flow of execution might change depends on the result evaluated by the condition. This type of statement is alsocalled decision making statements or control statements. This type of statement may skip some set of statements based on the condition.

Logical Conditions Supported by Python

- Equal to (==) Eg: a == b
- Not Equal (!=)Eg : a != b
- Greater than (>) Eg: a > b
- Greater than or equal to (>=) Eg: a >= b
- Less than (<) Eg: a < b
- Less than or equal to (<=) Eg: a <= b

Table 6 Indentation (Structure of C- Program Vs Python)

C Program	Python
-----------	--------

<pre>x = 500 y = 200 if (x > y) { printf("x is greater than y") } else if(x == y) { printf("x and y are equal") } else { printf("x is less than y") }</pre>	<pre>x = 500 y = 200 if x > y: print("x is greater than y") elif x == y: print("x and y are equal") else: print("x is less than y")</pre> ↑ Indentation (At least one White Space instead of curly bracket)
--	---

Structure of C- Program Vs Python

To represent a block of statements other programming languages like C, C++ uses “{ ...}” curly – brackets , instead of this curly braces python uses indentation using white space which defines scope in the code. The example given below shows the difference between usage of Curly bracket and white space to represent a block of statement.

Without proper Indentation:

```
x = 500
y = 200
if x > y:
print("x is greater than y")
```

In the above example there is no proper indentation after if statement which will lead to Indentation error.

If statement :

The 'if' statement is written using "if" keyword, followed by a condition. If the condition is true the block will be executed. Otherwise, the control will be transferred to the first statement after the block.

Syntax:

```
if<Boolean>:
```

```
    <block>
```

In this statement, the order of execution is purely based on the evaluation of boolean expression.

Example:

```
x = 200
y = 100
if x > y:
    print("X is greater than Y")

print("End")
```

Output :

X is greater than Y

End

In the above the value of x is greater than y, hence it executed the print statement whereas in the below example x is not greater than y hence it is not

executed the first print statement

```
x = 100
y = 200
if x > y:
    print("X is greater than Y")

print("End")
```

Output :

End

Elif

The **elif** keyword is useful for checking another condition when one condition is false.

Example

```
mark = 55

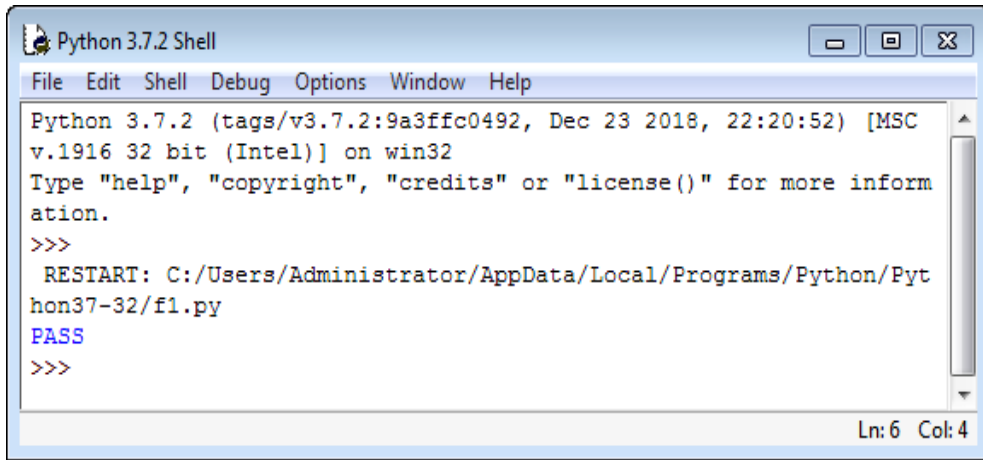
if (mark >=75):

    print("FIRST CLASS")
```

```
elif mark >= 50:
```

```
print("PASS")
```

Output :



```
Python 3.7.2 Shell
File Edit Shell Debug Options Window Help
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22:20:52) [MSC
v.1916 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more inform
ation.
>>>
RESTART: C:/Users/Administrator/AppData/Local/Programs/Python/Pyt
hon37-32/f1.py
PASS
>>>
```

Fig20 **elif** keyword

In the above the example, the first condition (mark >=75) is false then the control is transferred to the next condition (mark >=50), Thus, the keyword **elif** will be helpful for having more than one condition.

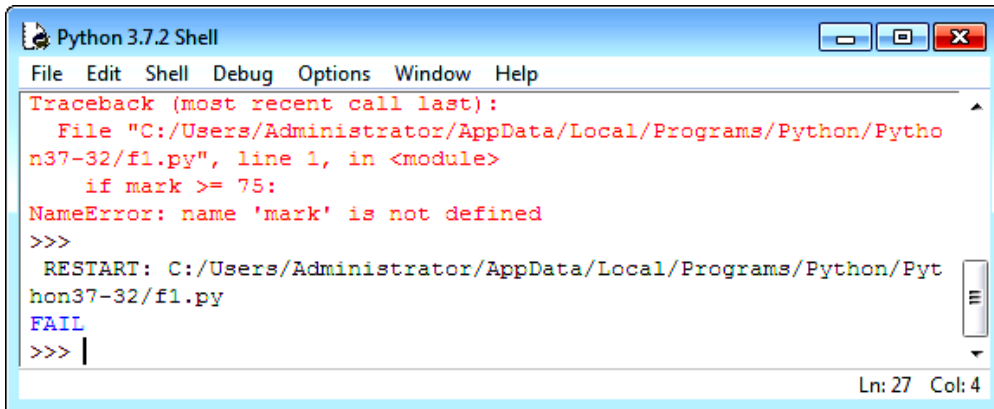
Else

The **else** keyword will be used as a default condition. i.e. When there are many conditions, when the **if-condition** is not true and all **elif-conditions** are also not true, then **else** part will be executed..

Example

```
mark = 10

if mark >= 75:
    print("FIRST CLASS")
elif mark >= 50:
    print("PASS")
else:
    print("FAIL")
```



```
Python 3.7.2 Shell
File Edit Shell Debug Options Window Help
Traceback (most recent call last):
  File "C:/Users/Administrator/AppData/Local/Programs/Python/Python37-32/f1.py", line 1, in <module>
    if mark >= 75:
NameError: name 'mark' is not defined
>>>
RESTART: C:/Users/Administrator/AppData/Local/Programs/Python/Python37-32/f1.py
FAIL
>>> |
Ln: 27 Col: 4
```

Fig 21 else keyword

In the example above, condition 1 and condition 2 fail. None of the preceding condition is true. Hence, the **else** part is executed.

Iterative Statements

Sometimes certain section of the code (block) may need to be repeated again and again as long as certain condition remains true. In order to achieve this, the iterative statements are used. The number of times the block needs to be repeated is controlled by the test condition used in that statement. This type of statement is also called as the “Looping Statement”. Looping statements add a surprising amount of new power to the program.

Need for Looping / Iterative Statement

Suppose the programmer wishes to display the string “Sathyabama !...” 150 times. For this, one can use the print command 150 times.

```
print("Sathyabama  
!...")  
print("Sathyabama  
!...")  
.....  
.....
```

The above method is somewhat difficult and laborious. The same result can be achieved by a loop using just two lines of code.(As below)

```
for count in  
range(1,150) :  
    print ("Sathyabama
```

Types of looping statements

- 1) **for** loop
- 2) **while** loop

The ‘for’ Loop

The **for** loop is one of the powerful and efficient statements in python which is used very often. It specifies how many times the body of the loops needs to be executed. For this reason it uses control variables which keep tracks, the count of execution. The general syntax of a ‘for’ loop looks as below:

```
for<variable>in range (A,B):  
  
    <body of the loop >
```

Flow Chart:

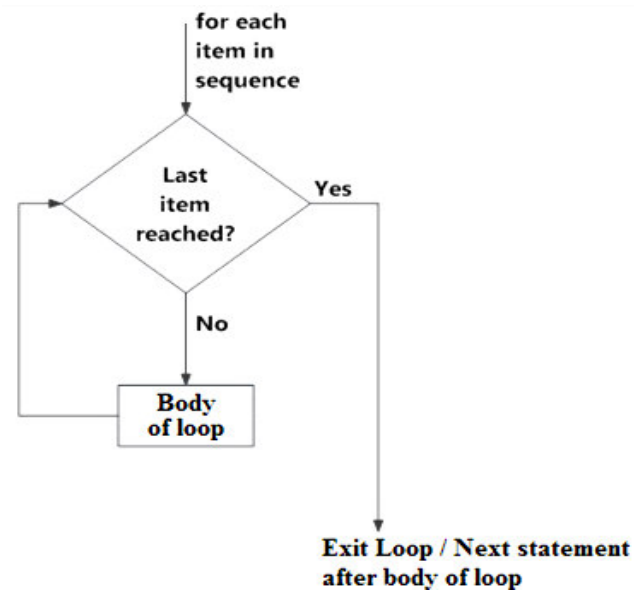


Fig 22 for Flow Chart

Example 1: To compute the sum of first n numbers (i.e. $1 + 2 + 3 + \dots + n$)

```
# Sum.py
```

```
total = 0
```

```
n = int (input ("Enter a Positive Number"))
```

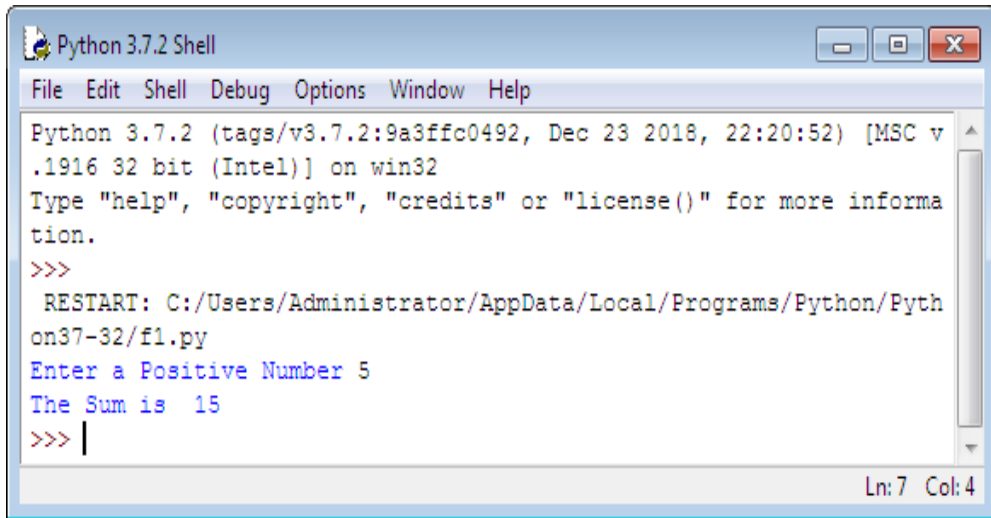
```
for i in range(1,n+1):
```

```
total = total + i
```

```
print ("The Sum is ", total)
```

Note: Why (n+1)? Check in table given below.

Output:



```
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22:20:52) [MSC v.1916 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/Users/Administrator/AppData/Local/Programs/Python/Python37-32/f1.py
Enter a Positive Number 5
The Sum is 15
>>> |
```

Fig 23statement $\text{total} = \text{total} + i$

In the above program, the statement $\text{total} = \text{total} + i$ is repeated again and again ‘n’ times. The number of execution count is controlled by the variable ‘i’. The range value is specified earlier before it starts executing the body of loop. The initial value for the variable i is 1 and final value depends on ‘n’. You may also specify any constant value.

The **range()** Function:

The **range()** function can be called in three different ways based on the number of parameters. All parameter values must be integers.

Table 7 range()

Type	Example	Explanation
------	---------	-------------

range(end)	for i in range(5): print(i) Output : 0,1,2,3,4	This is begins at 0.Increments by 1. End just before the value of end parameter.
range(begin,end)	for i in range(2,5): print(i) Output : 2,3,4	Starts at begin, End before end value, Increment by 1
range(begin,end,step)	for i in range(2,7,2) print(i) Output : 2,4,6	Starts at begin, End before end value, increment by step value

Example:To compute Harmonic Sum (ie: $1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots \frac{1}{n}$)

```
# harmonic.py
```

```
total = 0
```

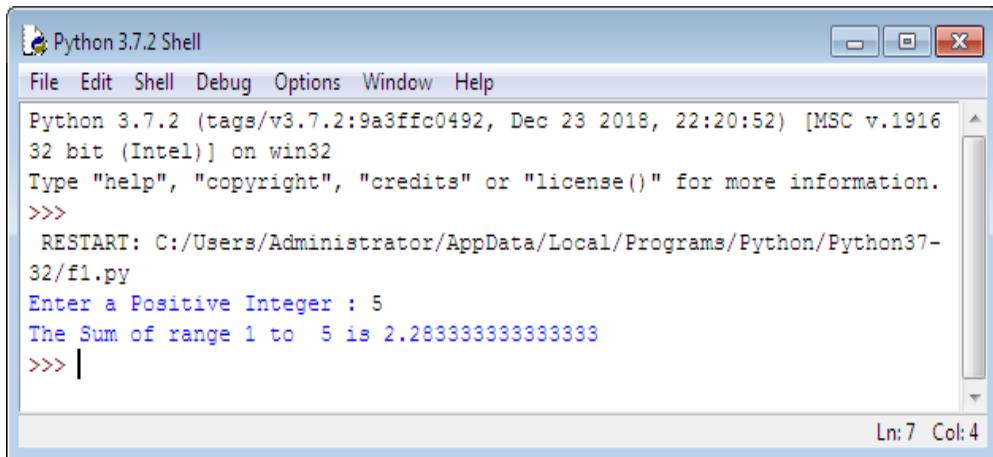
```
n= int(input("Enter a Positive Integer:"))
```

```
for i in range(1,n+1):
```

```
total+= 1/i
```

```
print("The Sum of range 1 to ",n, "is", total)
```

Output:



```
Python 3.7.2 Shell
File Edit Shell Debug Options Window Help
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22:20:52) [MSC v.1916
32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/Users/Administrator/AppData/Local/Programs/Python/Python37-
32/f1.py
Enter a Positive Integer : 5
The Sum of range 1 to 5 is 2.2833333333333333
>>> |
```

Fig 24compute Harmonic Sum

Example:

Factorial of a number "n"

```
n= int(input("Enter a Number :"))
```

```
factorial = 1
```

Initialize factorial value by 1

To verify whether the given number is negative / positive / zero

```
if n < 0:
```

```
print("Negative Number , Enter valid Number !...")
```

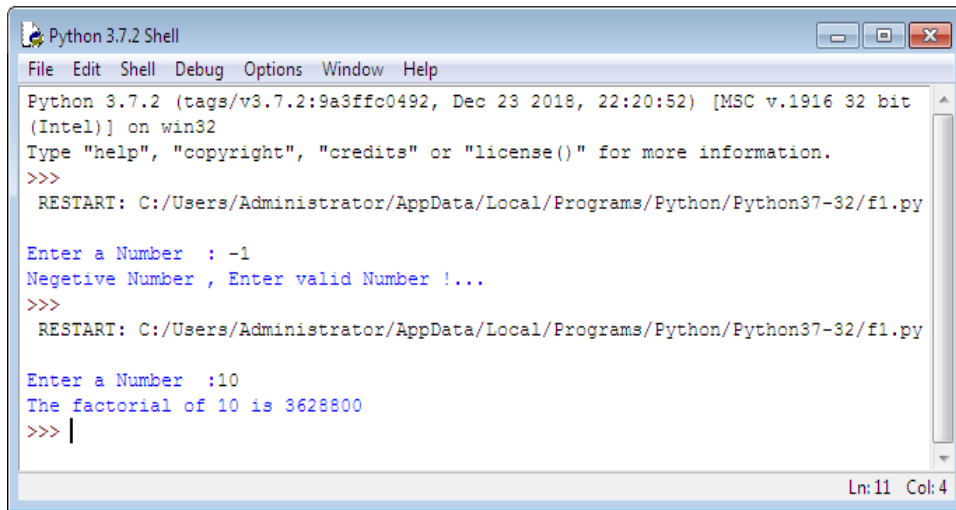
```
elif n == 0:
```

```
print("The factorial of 0 is 1")
```

```
else:
```

```
for i in range(1,n + 1):  
    factorial = factorial*i  
print("The factorial of" ,n, "is", factorial)
```

Output:



```
Python 3.7.2 Shell  
File Edit Shell Debug Options Window Help  
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22:20:52) [MSC v.1916 32 bit  
(Intel)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>>  
RESTART: C:/Users/Administrator/AppData/Local/Programs/Python/Python37-32/f1.py  
  
Enter a Number : -1  
Negative Number , Enter valid Number !...  
>>>  
RESTART: C:/Users/Administrator/AppData/Local/Programs/Python/Python37-32/f1.py  
  
Enter a Number :10  
The factorial of 10 is 3628800  
>>> |  
Ln:11 Col:4
```

Fig 25factorial

The **while** Loop

The **while** loop allows the program to repeat the body of a loop, any number of times, when some condition is true. The drawback of **while** loop is that, if the condition is not proper it may lead to infinite looping. So the user has to carefully choose the condition in such a way that it will terminate at a particular stage.

Flow Chart:

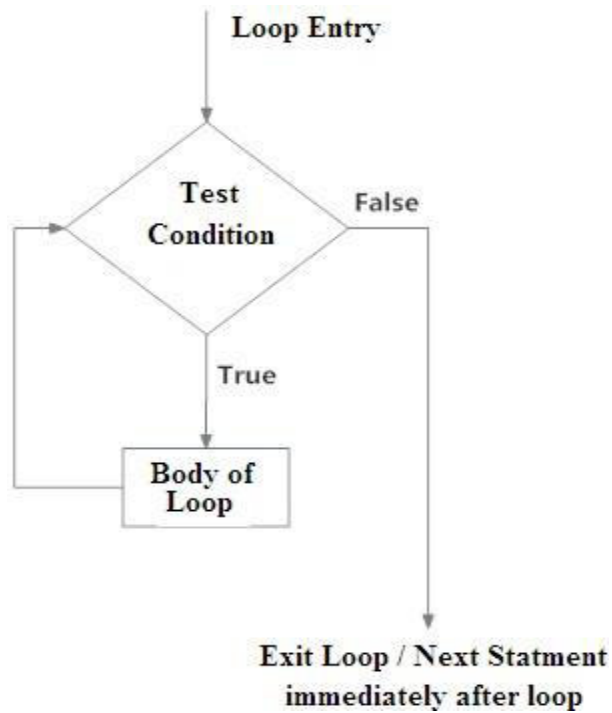


Fig 26 Flow chart while Loop

Syntax:

```
while (condition):  
    <body of the loop>
```

In this type of loop, The execution of the loop body is purely based on the output of the given condition. As long as the condition is TRUE or in other words until the condition becomes FALSE the program will repeat the body of loop.

Table 8 Example

Valid Example	Invalid Example
<pre>i = 10 while i < 15 : print(i) i = i + 1</pre> Output : 10,11,12,13,14	<pre>i = 10 while i < 15 : print(i)</pre> Output : 10,10,10,10..... Indeterminate number of times

Example: Program to display Fibonacci Sequence

```
# Program to Display Fibonacci Sequence based on number of terms
n
n = int(input("Enter number of terms in the sequence you want to
display"))
# n1 represents -- > first term and n2 represents --> Second term
n1 = 0
```

```

n2 = 1
count = 0
# count -- To check number of terms
if n <= 0:          # To check whether valid number of terms
    print ("Enter a positive integer")
elif n == 1:
    print("Fibonacci sequence up to",n,":")
    print(n2)
else:
    print("Fibonacci sequence of ",n, " terms :")
    while count < n:
        print(n1,end=' , ')
        nth = n1 + n2
        n1 = n2
        n2 = nth
    count = count + 1

```

Input / Output Statement:

Programmer often has a need to interact with users, either to get data or to provide some sort of result.

For Example: In a program to add two numbers, first the program needs to have an input of two numbers (The numbers which they prefer to add) and after processing, the output should be displayed. So to get the input of two

numbers, the program need to have an Input Statement and in order to display the result i.e. the sum of two numbers, it needs to have an Output Statement.

Input Statement: Helpful to take input from the user through input devices like keyboard. In Python, the standard input function is 'input()'

The syntax for input function is as follows:

input()

However, to get an input by prompting the user, the following form is used:

input('prompt')

where `prompt` is the string, which programmer wish to display on the screen to give more clarity about the input data. It is optional.

Example:

```
>>>num = input('Enter a number: ')
```

The above statement will wait till the user, enters the input value.

Output:

```
Enter a number:
```

```
>>>num
```

```
'10' # Input data entered by the user
```

Output Statement:

The output statement is used to display the output in the standard output devices like monitor (screen).The standard output function “print()” is used.

Syntax:

```
print('prompt')
```

where `prompt` is the string, which programmer wish to display on the screen

Example 1:

```
print('Welcome to the Python World !')
```

Output:

Welcome to the Python World !

Example 2:

```
X = 5
```

```
print ('The value of a is', X)
```

Output:

The value of X is 5

Example 3:

```
print(1,2,3,4)
```

Output: 1 2 3 4

Example 4:

```
print(100,200,300,4000,sep='*')
```

Output:

100*200*300*4000

Example 5:

```
print(1,2,3,4,sep='#',end='&')
```

Output:

1#2#3#4&

Output formatting

Sometimes we would like to format our output to make it look attractive. This can be done by using the `str.format()` method. This method is visible to any string object.

```
>>>x = 5; y = 10
>>>print('The value of x is {} and y is {}'.format(x,y))
The value of x is5and y is10
```

42

Here, the curly braces `{}` are used as placeholders. We can specify the order in which they are printed by using numbers (tuple index).

```
print('I love {0} and {1}'.format('bread','butter'))
```

```
print('I love {1} and {0}'.format('bread','butter'))
```

Output

I love

bread
and
butterI
love
butter
and
bread

We can even use keyword arguments to format the string.

```
>>> print('Hello {name}, {greeting}'.format(greeting =  
'Goodmorning', name = 'John'))
```

Hello John, Goodmorning